

Agenda

- Functional Interfaces
- Anonymous Inner Classes
- Lambda Expressions
- Method references
- Stream Programming

Built-in functional interfaces

- New set of functional interfaces given in java.util.function package.
 - `Predicate<T>`: test: T -> boolean
 - `Function<T,R>`: apply: T -> R
 - `BiFunction<T,U,R>`: apply: (T,U) -> R
 - `UnaryOperator<T>`: apply: T -> T
 - `BinaryOperator<T>`: apply: (T,T) -> T
 - `Consumer<T>`: accept: T -> void
 - `Supplier<T>`: get: () -> T
- For efficiency primitive type functional interfaces are also supported e.g. `IntPredicate`, `IntConsumer`, `IntSupplier`, `IntToDoubleFunction`, `ToIntFunction`, `ToIntBiFunction`, `IntUnaryOperator`, `IntBinaryOperator`.

Anonymous Inner Class

- Creates a new class inherited from the given class/interface and its object is created.
- If in static context, behaves like static member class. If in non-static context, behaves like non-static member class.
- Along with Outer class members, it can also access (effectively) final local variables of the enclosing method.

```
// (named) local class
class EmpnoComparator implements Comparator<Employee> {
    public int compare(Employee e1, Employee e2) {
        return e1.getEmpno() - e2.getEmpno();
    }
}
Arrays.sort(arr, new EmpnoComparator());    // anonymous obj of local class
```

```
// Anonymous inner class
Comparator<Employee> cmp = new Comparator<Employee>() {
    public int compare(Employee e1, Employee e2) {
        return e1.getEmpno() - e2.getEmpno();
    }
};
Arrays.sort(arr, cmp);
```

```
// Anonymous object of Anonymous inner class.
Arrays.sort(arr, new Comparator<Employee>() {
    public int compare(Employee e1, Employee e2) {
        return e1.getEmpno() - e2.getEmpno();
    }
});
```

Lambda expressions

- Traditionally Java uses anonymous inner classes to compact the code.
- For each inner class separate .class file is created.
- However code is complex to read and un-efficient to execute.
- Lambda expression is short-hand way of implementing functional interface.
- It Uses the instruction invokedynamic for the execution
- hence there is no separate .class file created for the lambda expressions
- Its argument types may or may not be given. The types will be inferred.
- Lambda expression can be single liner (expression not statement) or multi-liner block { ... }.

```
// Anonymous inner class
Arrays.sort(arr, new Comparator<Emp>() {
    public int compare(Emp e1, Emp e2) {
        int diff = e1.getEmpno() - e2.getEmpno();
        return diff;
    }
});
```

```
// Lambda expression -- multi-liner
Arrays.sort(arr, (Emp e1, Emp e2) -> {
    int diff = e1.getEmpno() - e2.getEmpno();
    return diff;
});
```

```
// Lambda expression -- multi-liner -- Argument types inferred
Arrays.sort(arr, (e1, e2) -> {
    int diff = e1.getEmpno() - e2.getEmpno();
    return diff;
});
```

```
// Lambda expression -- single-liner -- with block { ... }
Arrays.sort(arr, (e1, e2) -> {
    return e1.getEmpno() - e2.getEmpno();
});
```

```
// Lambda expression -- single-liner  
Arrays.sort(arr, (e1,e2) -> e1.getEmpno() - e2.getEmpno()));
```

- Practically lambda expressions are used to pass as argument to various functions.
- Lambda expression enable developers to write concise code (single liners recommended).

Non-capturing lambda expression

- If lambda expression result entirely depends on the arguments passed to it, then it is non-capturing (self-contained).

```
BinaryOperator<Integer> op1 = (a,b) -> a + b;  
testMethod(op);
```

```
static void testMethod(BinaryOperator<Integer> op) {  
    int x=12, y=5, res;  
    res = op.apply(x, y); // res = x + y;  
    System.out.println("Result: " + res)  
}
```

- In functional programming, such functions/lambda expressions are referred as pure functions.

Capturing lambda expression (Demo04)

- If lambda expression result also depends on additional variables in the context of the lambda expression passed to it, then it is capturing.

```
int c = 2; // must be effectively final  
BinaryOperator<Integer> op = (a,b) -> a + b + c;  
testMethod(op);
```

```
static void testMethod(BinaryOperator<Integer> op) {  
    int x=12, y=5, res;  
    res = op.apply(x, y); // res = x + y + c;  
    System.out.println("Result: " + res);  
}
```

- Here variable `c` is bound (captured) into lambda expression. So it can be accessed even out of scope (effectively). Internally it is associated with the method/expression.
- In some functional languages, this is known as Closures.

Method references

- lambda expression is an short-hand implementation of Single Abstract Method (Functional Interface)
- Method reference is short-hand of lambda-expression
- If lambda expression involves single method call, it can be shortened by using method reference.
- Method references are converted into instances of functional interfaces.
- Method reference can be used for class static method, class non-static method, object non-static method or constructor.

Java 8 Streams

- Java 8 Stream is NOT IO streams.
- java.util.stream package.
- Streams follow functional programming model in Java 8.
- The functional programming is based on functional interface (SAM).
- Number of predefined functional interfaces added in Java 8. e.g. Consumer, Supplier, Function, Predicate, ...
- Lambda expression is short-hand way of implementing SAM -- arg types & return type are inferred.
- Java streams represents pipeline of operations through which data is processed.
- Stream operations are of two types

1. Intermediate operations: Yields another stream.

- intermediate operations are again classified as

1. stateless operation

- filter(), map(), flatMap(), limit(), skip()

2. stateful operation

- sorted(), distinct()

2. Terminal operations: Yields some result.

- reduce()
- forEach()
- collect(), toArray()
- count(), max(), min()
- Stream operations are higher order functions (take functional interfaces as arg).

Java stream characteristics

1. No storage: Stream is an abstraction. Stream doesn't store the data elements. They are stored in source collection or produced at runtime.
2. Immutable: Any operation doesn't change the stream itself. The operations produce new stream of results.
3. Lazy evaluation: Stream is evaluated only if they have terminal operation. If terminal operation is not given, stream is not processed.
4. Not reusable: Streams processed once (terminal operation) cannot be processed again.

Stream creation

- Collection interface: `stream()` or `parallelStream()`
- Arrays class: `Arrays.stream()`
- Stream interface: static `of()` method
- Stream interface: static `generate()` method
- Stream interface: static `iterate()` method
- Stream interface: static `empty()` method
- nio Files class: `static Stream<String> lines(filePath)` method

Stream creation

- Collection interface: `stream()` or `parallelStream()`

```
List<String> list = new ArrayList<>();  
// ...  
Stream<String> strm = list.stream();
```

- Arrays class: `Arrays.stream()`

```
Double arr[] = {1.1,2.2,3.3,4.4,5.5,6.6,7.7,8.8,9.9};  
Stream<Double> strm = Arrays.stream(arr);
```

- Stream interface: static `of()` method

```
Stream<Integer> strm = Stream.of(arr);
```

- Stream interface: static `generate()` method

- `generate()` internally calls given Supplier in an infinite loop to produce infinite stream of elements.

```
Stream<Double> strm = Stream.generate(() -> Math.random()).limit(25);
```

```
Random r = new Random();  
Stream<Integer> strm = Stream.generate(() -> r.nextInt(1000)).limit(10);
```

- Stream interface: static `iterate()` method

- `iterate()` start the stream from given (arg1) "seed" and calls the given UnaryOperator in infinite loop to produce infinite stream of elements.

```
Stream<Integer> strm = Stream.iterate(1, i -> i + 1).limit(10);
```

- Stream interface: static empty() method
- nio Files class: static Stream lines(filePath) method

SUNBEAM INFOTECH